

CTE - Common Table Expressions

Kiến thức tổng quan về CTE (Common Table Expressions) trong SQL

1. CTE là gì?

CTE (Common Table Expression) là một khối truy vấn tạm thời, có thể được tham chiếu trong một truy vấn SQL chính. CTE giúp viết truy vấn phức tạp theo cách rõ ràng, dễ đọc hơn bằng cách chia thành các phần có thể tái sử dụng.

2. Cú pháp tổng quát

```
WITH ten_cte AS (  
    SELECT ...  
    FROM ...  
    WHERE ...  
)  
SELECT ...  
FROM ten_cte  
WHERE ...;
```

Trong đó:

- **WITH**: khai báo CTE
- **ten_cte**: tên gọi tạm của bảng kết quả từ truy vấn
- CTE có thể sử dụng trong câu truy vấn chính như một bảng

3. CTE nhiều khối

Có thể khai báo nhiều CTE cùng lúc, ngăn cách bằng dấu phẩy:

```

WITH cte1 AS (
    SELECT ...
),
cte2 AS (
    SELECT ...
)
SELECT ...
FROM cte1
JOIN cte2 ON ...;

```

4. So sánh CTE với Subquery

Tiêu chí	CTE	Subquery
Đọc dễ	Rõ ràng, tách riêng	Có thể lồng sâu, khó đọc
Tái sử dụng	Có thể gọi nhiều lần	Không tái sử dụng
Tối ưu hóa	PostgreSQL thường tối ưu tốt	Đôi khi khó tối ưu khi lồng sâu

5. Ví dụ cơ bản

Ví dụ: Lấy top 3 sản phẩm có giá cao nhất

```

WITH top_products AS (
    SELECT *
    FROM products
    ORDER BY price DESC
    LIMIT 3
)
SELECT name, price
FROM top_products;

```

6. Ví dụ CTE dùng nhiều lần

```

WITH expensive_products AS (
    SELECT *
    FROM products
    WHERE price > 1000
)
SELECT COUNT(*) FROM expensive_products;

-- Hoặc dùng lại để join
SELECT p.*, c.name as category_name
FROM expensive_products p
JOIN categories c ON c.id = p.category_id;

```

7. CTE đệ quy

CTE cũng hỗ trợ đệ quy (recursive CTE), dùng trong các bài toán như cây phân cấp, tổ tiên – hậu duệ...

Ví dụ: tìm tất cả danh mục con của một danh mục cha

```

WITH RECURSIVE category_tree AS (
    SELECT id, name, parent_id
    FROM categories
    WHERE id = 1 -- danh mục gốc

    UNION ALL

    SELECT c.id, c.name, c.parent_id
    FROM categories c
    JOIN category_tree ct ON c.parent_id = ct.id
)
SELECT * FROM category_tree;

```

8. Một số lưu ý khi dùng CTE

- CTE **không lưu tạm trên đĩa**, mà thực thi như subquery (trừ khi ép materialize hoặc PostgreSQL tự xử lý)
- Trong PostgreSQL 12+, có thể dùng `MATERIALIZED` hoặc `NOT MATERIALIZED` để kiểm soát tối ưu hóa:

```
WITH MATERIALIZED cte AS (  
    SELECT ...  
)  
SELECT * FROM cte;
```

9. Khi nào nên dùng CTE

- Truy vấn phức tạp nhiều tầng
- Cần dùng cùng dữ liệu trung gian ở nhiều chỗ
- Tách logic tính toán để dễ maintain
- Viết báo cáo hoặc thống kê nhiều bước

10. Kết luận

CTE là công cụ cực kỳ mạnh để viết truy vấn SQL rõ ràng, có cấu trúc và dễ bảo trì. Trong hệ thống lớn hoặc cần báo cáo nhiều bước, CTE giúp quản lý truy vấn tốt hơn so với subquery thuần túy. Tuy nhiên, cần kiểm tra execution plan để đảm bảo CTE không gây overhead nếu bị tính lại nhiều lần hoặc không được planner tối ưu.